PROG6212 – BONUS ICE TASK

Student Number: St10447616 (Ammaarah Khan)

(Question: “Open IL disassembler, look at methods we did, click any 4, double click on it then explain in-depth explanation of the IL code”)

1. The ***AddTwoNumbers*** method is a public static method that returns an int. It declares one local integer V_0. The IL loads the two input arguments onto the evaluation stack (ldarg.0, ldarg.1), it uses add to sum them, and also stores the result into local V_0 (stloc.0). A short branch (br.s) goes to the return, which loads V_0 back onto the stack (ldloc.0) and returns it (ret).

Line-by-line code analysis for **AddTwoNumbers**:

- .method public ... - This line is the method header; It names the method, says it's public (which means that any code can call it), static (would belong to the class, not to a specific object), and int32 is the return type (an integer).
- .maxstack 2 - Rule for the method: the stack only needs to handle up to 2 things at the same time.
- .locals init ([0] int32 V_0) - Declares one local variable: the first local box V_0 is holding an integer.
- IL_0000: nop - No operation. It doesn't really do anything; mostly used as a placeholder or for debugging.
- IL_0001: ldarg.0 - Loads argument 0 onto the stack; It pushes the first input 'a' onto the top of the stack.
- IL_0002: ldarg.1 - Loads argument 1 onto the stack.
- IL_0003: add - Pop two numbers, add them, then push result; Pops b and a, calculates a + b, pushes the sum.
- IL_0004: stloc.0 - Store top of stack into local variable V_0; basically pop the sum from the top and put it into the box V_0. (So V_0 = a + b.)
- IL_0005: br.s IL_0007 - Jump to instruction IL_0007; This just moves control forward to the return.

- IL_0007: ldloc.0 - Load the value stored in local V_0 onto the stack and prepares the return value by putting it back on the stack.
- IL_0008: ret - Return the top-of-stack value (the sum) to the caller. Th method ends and hands back the integer.

2. ***CreateGreeting*** sets a local prefix to "Hello", then it loads a prefix, a space " ", the parameter name, and "!" onto the evaluation stack and calls System.String.Concat (the 4-argument overload). The concatenated string is stored in local V_1, loaded, and returned. Header — method CreateGreeting returns string and accepts string name.

Line-by-line code analysis for ***CreateGreeting***:

- .maxstack 4 - the stack may need up to 4 items at once (pass 4 strings into Concat).
- .locals init ([0] string prefix, [1] string V_1) - two local boxes: prefix (index 0) and V_1 (index 1). prefix will store "Hello" and V_1 will hold the final greeting.
- IL_0000: nop - placeholder.
- IL_0001: ldstr "Hello" - push the text "Hello" onto the stack.
- IL_0006: stloc.0 - pop that "Hello" off the stack and store into prefix (box V_0).
- IL_0007: ldloc.0 - loads prefix back onto the stack (the "Hello" is back on the stack again).
- IL_0008: ldstr " " - push a space " " (so far stack: " ", "Hello" on top).
- IL_000d: ldarg.0 - push the input name onto the stack.
- IL_000e: ldstr "!" - push "!" onto the stack — now there are four strings ready.
- IL_0013: call System.String::Concat(string,string,string,string) - call the .NET method that concatenates 4 strings. It pops the 4 strings and pushes back the single combined string.
- IL_0018: stloc.1 - store the concatenated result into V_1 (final result box).
- IL_0019: br.s IL_001b - jump to final load/return.
- IL_001b: ldloc.1 - load result onto the stack.

- IL_001c: ret - return the string.

3. ***CountToFive*** initializes local i to 0 and uses a loop: it formats and prints i + 1 with a comma, increments i, and repeats as long as i < 5. Comparison clt produces 1 when i < 5 and that value controls brtrue.s to repeat the loop; after the loop it prints a blank line and returns. box is used so the integer can be passed as object to String.Format.

Line-by-line code analysis for ***CountToFive***:

- .locals init ([0] int32 i, [1] bool V_1) — two local boxes: i (an integer) and V_1 (a boolean used for the loop test).
- IL_0001: ldc.i4.0 - Load constant integer 0 onto the stack. (Push 0.)
- IL_0002: stloc.0 - Store that 0 into local i. So i = 0.
- IL_0003: br.s IL_0023 - Jump to the loop condition check. This will set up the C-style while structure: first check, then body.

- Loop body (starts IL_0005):
  - IL_0006: ldstr "{0}," - push the format string "{0}," onto stack; This string is used by String. Format- {0} will be replaced by the number.
  - IL_000b: ldloc.0 - push current i onto stack.
  - IL_000c: ldc.i4.1 - push integer 1 (we need to print i + 1 not i).
  - IL_000d: add - pop i and 1, push i+1.
  - IL_000e: box System.Int32 - Box the integer.

Basically, we'd be changing the raw number into an “object” package so it can be used by methods that expect object (like String.Format).

- IL_0013: call System.String::Format(string, object) - call String.Format("{0},", object) which produces a string like "1," or "2,".
- IL_0018: call System.Console::WriteLine(string) - prints formatted string to the console.
- IL_001f: ldloc.0 - load i.

- IL_0020: ldc.i4.1 - push 1.
- IL_0021: add - compute i + 1 (this increments).
- IL_0022: stloc.0 - store back into i. So i = i + 1.
- IL_0023: ldloc.0 - push i.
- IL_0024: ldc.i4.5 - push 5.
- IL_0025: clt - Compare less-than. It pops the two values and pushes 1 if i < 5, otherwise 0.
- IL_0027: stloc.1 - store the comparison result (0/1) into V_1 which is the boolean.
- IL_0028: ldloc.1 - loads the boolean.
- IL_0029: brtrue.s IL_0005 - If the boolean is true (non-zero), jump to the loop body. So while i < 5, the loop continues.
- IL_002b: call void System.Console::WriteLine() - prints a blank line.
- IL_0031: ret - return (method done).

4. ***CheckNumber*** computes number >= 0 by first checking number < 0 (clt) and then using ceq against 0 - the effect is that the stored boolean V_0 is true when number >= 0. brfalse.s tests that boolean: if false it jumps to the negative-case block, otherwise it formats and prints "{0} is positive". The negative block formats "{0} is negative". box is used to pass the integer as an object to String.Format.

Line-by-line code analysis for ***CheckNumber*****:**

- .locals init ([0] bool V_0) - a local boolean V_0 will hold the test result.
- IL_0001: ldarg.0 - push the method parameter number on the stack.
- IL_0002: ldc.i4.0 - push constant 0.
- IL_0003: clt - Compares Less Than: pops number and 0, and pushes 1 if number < 0, otherwise 0. So after this, if number was negative then the stack top is 1 but if number was zero or positive then the stack top is 0.
- IL_0005: ldc.i4.0 - push 0 again.
- IL_0006: ceq - Compare Equal: compares the top two items and pushes 1 if they are equal, otherwise 0. Here it checks whether the previous result (is number < 0?) is equal to 0.
- If clt pushed 1 (number < 0), comparing with 0 gives 0 (false).

- If clt pushed 0 (number >= 0), comparing with 0 gives 1 (true). So, the pair clt then ceq results in 1 when number >= 0, and 0 when number < 0.
- IL_0008: stloc.0 - store that boolean into V_0. So V_0 is true when number >= 0.
- IL_0009: ldloc.0 - load the boolean.
- IL_000a: brfalse.s IL_0026 - If the boolean is false, jump to IL_0026.
- If V_0 is false then number is negative so it jumps to the negative message, but If V_0 is true then continue to the positive message.
- IL_000d: ldstr "{0} is positive" - push the format string.
- IL_0012: ldarg.0 - push number.
- IL_0013: box System.Int32 - box number to object for String.Format.
- IL_0018: call String::Format(string, object) - get the message like "5 is positive".
- IL_001d: call Console::WriteLine(string) - print it.
- IL_0024: br.s IL_003e - skip the negative branch and jump to end.
- Negative message block (IL_0026): similar steps but with "{0} is negative".
- IL_003e: ret - return.